Community Intelligence with Metadata-Filtered Retrieval-Augmented Generation: Grounded Question Answering, Search and Explainable Recommendation for a Skill-Development Community

Mrs. Sonal Gore[1], Ishan Patil[1*], Pushkar Kirange[1], Aditya Atre[1], Aaryan Bhujang[1]
[1]Department of Computer Engineering, Pimpri Chinchwad College of Engineering,
Pimpri Chinchwad Education Trust's, Pune, 411044, Maharashtra, India
*Corresponding author(s). E-mail(s): sonal.gore@pccoepune.org

Abstract

Members of a skill-development community often need answers about chapters, curricula, events and other members, but that information sits in scattered records and web pages. A general-purpose large language model (LLM) does not know this private data and, when asked, tends to invent answers. We reviewed 36 documents from 2002 to 2025 (20 international conference papers, 8 journal papers and 8 preprints, books and technical documents) on retrieval-augmented generation (RAG), dense retrieval, cited generation, RAG evaluation and cold-start recommendation. In this literature, question answering, attribute-constrained search and recommendation are almost always separate systems. We built one platform that brings them together. A single metadata-filtered RAG backbone drives a chatbot that cites its sources, natural-language search with structured filters, explained cold-start recommendations and LLM-written summaries for administrators. The knowledge base holds 273 chunks from 29 pages of a public community website, 72 member records and 32 event records, stored in ChromaDB with all-MiniLM-L6-v2 embeddings. An answer is shown only if it cites a retrieved source; otherwise the system refuses. With the hosted gpt-oss-20b model, 16 of 20 automatically generated questions were answered, 94% of those answers were judged faithful, and mean relevance was 4.81/5. The other four were refused rather than guessed. Fifteen of sixteen functional tests passed; the failing one exposed an error-mapping bug in the LLM layer.

Keywords: Retrieval-Augmented Generation, Large Language Models, Semantic Search, Vector Database, Metadata Filtering, Grounded Generation, Hallucination, Explainable Recommendation, Cold Start, Community Intelligence.

1. Introduction

Many skill-development communities run free training centres in partner colleges and city chapters. The community that we have chosen works in four different fields: Cybersecurity, AI/ML, Cloud and DevOps, and Software Development. Each of these domain is categorized into beginner, intermediate and advanced level. Users might ask questions to the platform based on these four domains. This type of community generates information all the times with the four domains that are given. For example, information about Cloud and DevOps. What is currently going on in this domain? There are some of the narrow and broader aspects of these domains that are usually questioned by many of the users that will be answered by this platform.

An LLM is an obvious tool which can be used for this type of the problem because the users might ask the questions in the natural language that could be answered only by the chatbots. RAG based chatbots are something that works on the data which have been fed into it. But there are some of the limitations of it such as hallucinations [5]. So we have to make sure that the model does not hallucinate. So the system prompt is one thing that could be most favorable aspect for it. When the model is unsure about the answer, it should not directly hallucinate and should give the appropriate answer to the user. Retrieval-augmented generation (RAG) works around the problem by fetching relevant passages at question time and asking the model to answer from them [6].

In most such communities, the information is spread across spreadsheets, forms and web pages, and chapter leads end up answering routine questions by hand. Search does not help much. Keyword search and plain vector RAG rank passages by similarity and have no way to enforce a domain, chapter, level or date, so a question that combines these returns records that look plausible but are wrong. Recommending the professionals as well as the events is not the easy part because we have to make sure that the answers which the user wants depends on the experience of different users who have used this platform because the professionals and the events is a historical and a young growing community. Administrators from their part have the logs of the questions which are gone unanswered. And it doesn't see which users have given those questions.

Our system design provides one metadata-filtered RAG chatbot. The chatbot is filled with the information which are publicly available from the website and the community records. This is stored in the ChromaDB vector database after chunking the information. On top of this layer, there are four modules which are used. First one is the chatbot which has to cite a source or it has to refuse. A search module that converts a plain language query into structured filters. A recommender that scores the professionals on the attributes and explain the suggestions. And an insights module that

groups logged questions. Some of the questions may be refused by the chatbot due to lack of the information fed to the chatbot. Those questions go to the administrators. Once the administrator answers from their side, it would be directly logged into the database.

We began with a literature review, since the design choices above only make sense against what already exists. The review looks at RAG architectures, retrieval and vector indexing, grounding and evaluation, and recommendation. For each work we note the evaluation parameters and the limitations the authors themselves report, and from these we draw out the gaps an integrated platform would need to close. The rest of the paper is organised around three research questions:

- **RQ1:** Can a refusal-first RAG chatbot running on a free hosted or local LLM answer community questions with high faithfulness?
- **RQ2:** Does metadata filtering improve multi-constraint questions compared with plain vector retrieval?
- **RQ3:** Can one shared retrieval backbone support search, explainable recommendation and administrator insight without separate models?

The paper makes four contributions:

1. A review of 36 documents on RAG architectures, retrieval, grounding, evaluation and recommendation, with their evaluation parameters and open gaps (Sections 2 and 3).
2. The design and implementation of a community intelligence platform where one metadata-filtered RAG backbone serves a cited chatbot, filtered search, explained cold-start recommendation and administrator insights (Section 4).
3. An evaluation made up of sixteen functional tests, an automated RAG evaluation scored by an LLM judge, and a head-to-head comparison of plain and metadata-filtered retrieval on multi-constraint questions (Sections 5.1–5.5).
4. An analysis of where the system failed and what that means for small organisations that want a grounded assistant (Section 5.6).

2. Related work

We grouped the reviewed work into four areas: LLMs and hallucination; RAG architectures; retrieval, embedding and vector indexing, together with methods for grounding and evaluating answers; and recommendation in cold-start settings, including how recommendations are explained.

2.1 Bibliography Analysis

Figure 1 breaks down the reviewed documents by source.

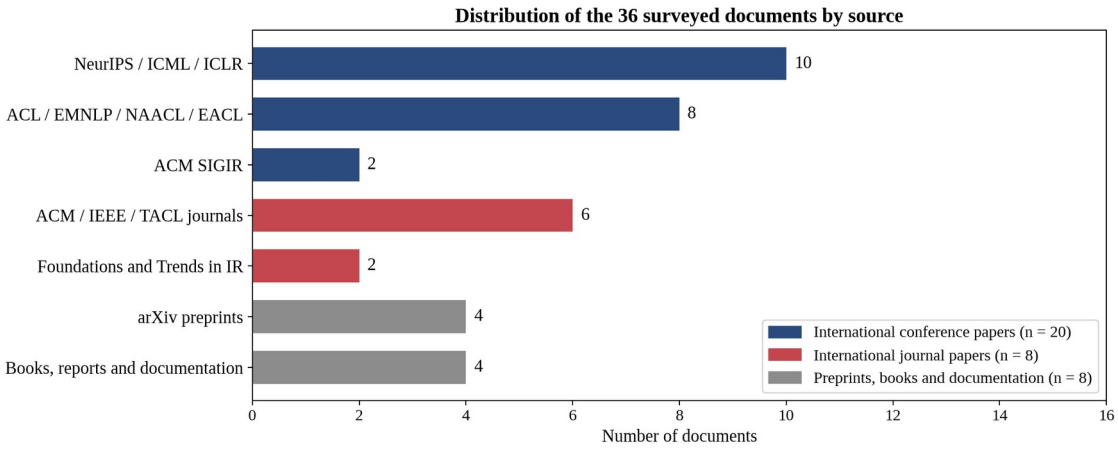


Figure 1. Bibliography analysis of the 36 surveyed documents

Table 1 counts the 36 documents by type (20 international conference papers, 8 international journal papers, and 8 preprints, books and technical documents). Table 2 counts them by year.

Table 1. Number of documents referred on the basis of type

Document Type	# Documents
International conference papers — NeurIPS / ICML / ICLR	10
International conference papers — ACL / EMNLP / NAACL / EACL	08
International conference papers — ACM SIGIR	02

Document Type	# Documents
International journal papers — ACM Computing Surveys, TACL, IEEE TPAMI / TBD / TKDE	06
International journal papers — Foundations and Trends in Information Retrieval	02
arXiv preprints	04
Books, technical reports and online documentation	04
Total (20 conference + 8 journal + 8 other)	**36**

Table 2. Distribution of documents based on the publication year

Publication Year	Conference	Journal	Other	Total
2025	0	0	1	01
2024	2	2	1	05
2023	2	2	1	05
2022	2	0	0	02
2021	4	1	1	06
2020	6	2	0	08
2019	2	0	1	03
≤ 2018	2	1	1	04
Online documentation	0	0	2	02
Total	**20**	**8**	**8**	**36**

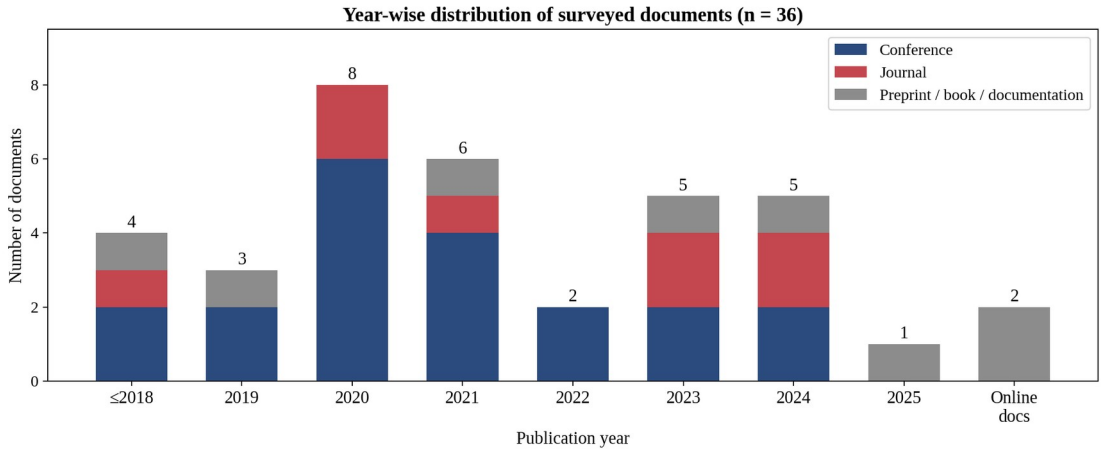


Figure 2. Number of documents referred on the basis of publication year (n = 36)

Most of this literature is new. Of the 34 documents with a publication year, 27 appeared in 2020 or later, which is when both RAG and dense passage retrieval were introduced. The four older documents (2018 or earlier) are the foundations the newer methods rest on: probabilistic ranking, cold-start recommendation, content-based recommendation and the Transformer. The remaining two are the online documentation of software we used.

2.2 Survey of Large Language Models and Hallucination

The Transformer of Vaswani et al. (2017) replaced recurrence with self-attention, and that change is what made pre-training at scale practical [1]. BERT (Devlin et al., 2019) then showed that a bidirectionally pre-trained encoder transfers to a wide range of understanding tasks [2]. Perspective of the generation side, Brown et al. (2020) found that a large decoder can pick up a new task from a bunch of examples in the prompt [3] and Ouyang et al. (2022) used the feedback from humans to make the model follow instructions [4].

Ji et al. (2023) gives the survey that hallucination in natural language generation are split into two kinds. Intrinsic is the first one where the output contradicts the source and extrinsic where it simply cannot be checked. The remedies are retrieval and grounding that they have identified [5].

2.3 Survey of Retrieval-Augmented Generation Architectures

RAG itself is introduced from the paper by Lewis et al. (2020), who joined a dense passage retriever to a BART generator over a Wikipedia index and trained the above components together. The model gave a new state-of-the-art model on the questions which are fed. Those are the question answering. Although every passage is scored independently [6]. REALM (Guu et al., 2020) learns to retrieve during language model pretraining [8]. Fusion-in-Decoder one at a time takes the passage, encodes it, and combines them in the decoder [9]. And RETRO retrieves the information from a very large database which lets a much smaller model keep up with far larger ones [10].

Training is not always needed. Ram et al. (2023) placed the retrieved chunks in front of a frozen LLM and still saw larger gains. This in-context RALM setting is the one our system uses [11]. Retrieval also lowers hallucination in knowledge-grounded dialogue [12]. Self-RAG goes further and trains the generator to make it decide for itself at what time it should retrieve and to validate its own output [13]. Gao et al. sort the field into naive and advanced RAG [14]. One finding that bears directly on our design comes from Liu et al. (2024). Models use information at the start and end of a long context better than information in the middle [15].

2.4 Survey of Retrieval, Embedding and Vector Indexing

BM25 which is described framework by Robertson and Zaragoza (2009) is the grounded probabilistic relevance framework which is still the baseline used for the embeddings [16]. Dense Passage Retrieval (DPR) by Karpukhin et al. (2020) uses two BERT encoders trained with in-batch negatives and clearly outperform BM25 on top-20 retrieval accuracy [17]. Sentence-BERT (Reimers and Gurevych, 2019) produces sentence embeddings that can be compared directly with cosine similarity [18]. MiniLM (Wang et al., 2020) is a distilled Transformer that keeps most of its teacher's accuracy at much lower cost; the all-MiniLM-L6-v2 encoder in our system is built on it [19].

Some methods trade speed for precision. ColBERT (Khattab and Zaharia, 2020) keeps one embedding per token and matches them at query time, a scheme called late interaction [20], while Nogueira and Cho (2019) re-rank candidates with a BERT cross-encoder [21]. Dense retrievers are not always robust, though. On the BEIR benchmark, Thakur et al. (2021) found that they do not always carry over well to new domains, whereas BM25 holds up as a zero-shot baseline [22].

2.5 Survey of Grounding, Citation and Evaluation

WebGPT (Nakano et al., 2021) was trained to browse the web and to quote the references behind its answers [25]. Gao et al. (2023) build ALCE which is a benchmark that scores recall and precision to measure this kind of behavior. Even the strongest model they tested left roughly half of the long-form answers without full citation support [26]. KILT (Petroni et al., 2021) puts many knowledge intensive tasks under a single benchmark and give the scores alongside the answer [27].

Labelled data is rare in small domains, so reference-free evaluation is attractive. RAGAS (Es et al., 2024) asks an LLM to score faithfulness, answer relevance and context relevance without any gold answers [28]. Zheng et al. (2023) suggests that strong LLM judges agree with the human preferences as same as two humans agreeing on each other. They also document position and verbosity biases [29]. Our reading of these two papers is that an LLM judge is a reasonable choice here, but only if it is checked against people.

2.6 Survey of Recommendation for Cold-Start Settings

A new user has no history in the system. This is called the cold-start problem. Schein et al. (2002) described this problem and gave metrics to measure it [7]. Lops et al. (2011) reviewed content-based recommenders [30]. These systems match item details with the user profile. So they do not need any past interactions. Zhang and Chen (2020) studied how recommendations are explained [31]. Some explanations show what the model really did. Others are written later, after the ranking is done. Zhao et al. (2024) reviewed recommenders that use LLMs [32]. These can explain their picks in plain language. But most of them still need a long interaction history.

Table 3 compares the main papers we reviewed. It shows the focus, method, parameters, results and limits of each one.

Table 3. Detailed comparison of surveyed works

Paper	Research Focus	Implementation Method	Parameters Used	Performance Metrics	Limitations
RAG (Lewis et al., NeurIPS 2020)	Knowledge-intensive generation	DPR retriever + BART generator, trained end to end	Wikipedia index; top-k retrieved passages	State of the art on open-domain QA (exact match)	Needs fine-tuning; no attribute constraints
DPR (Karpukhin et al., EMNLP 2020)	Dense passage retrieval	Dual BERT encoders, in-batch negatives	21 M Wikipedia passages; FAISS index	Top-20 accuracy well above BM25	Needs labelled QA pairs; similarity only

Paper	Research Focus	Implementation Method	Parameters Used	Performance Metrics	Limitations
In-Context RALM (Ram et al., TACL 2023)	Retrieval for frozen LLMs	Retrieved documents prepended to the prompt	BM25 retrieval; retrieval stride and query length	Large perplexity gains without training	Language modelling only; no citations
Self-RAG (Asai et al., ICLR 2024)	Adaptive retrieval and self-critique	Generator fine-tuned with reflection tokens	Llama 2 7B / 13B	Better factuality and citation accuracy	Requires training the generator
ALCE (Gao et al., EMNLP 2023)	Generating text with citations	Benchmark with entailment-based citation scoring	Citation recall and precision	Best models lack full citation support on ~50% of long answers	Benchmark, not a deployable system
RAGAS (Es et al., EACL 2024)	Reference-free RAG evaluation	LLM-scored faithfulness and relevance	Faithfulness, answer and context relevance	Close agreement with human judgements	Depends on judge quality
LLM-as-a-Judge (Zheng et al., NeurIPS 2023)	Automatic evaluation	Strong LLM grades answers	Pairwise and single-answer grading	Over 80% agreement with humans	Position and verbosity bias
LLM recommenders (Zhao et al., TKDE 2024)	Recommendation with LLMs	Prompting, in-context learning, tuning	Interaction histories	Natural-language explanations	Weak in cold start; explanations post hoc

3. Common Findings from Literature Review

3.1 Results and Common Methodology

The below figure summarizes the pipeline that most of the RAG systems use from building the information to evaluating the responses.

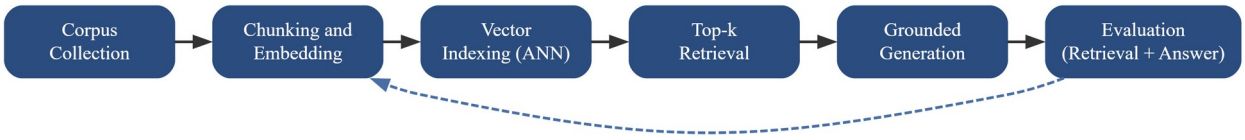


Figure 3. Common flow diagram of the RAG pipelines

The gist of it is like a domain information is collected, converted to the vector embeddings using vector embedding model and stored into the vector database. That vector database contains the information along with the metadata so that whenever user query comes, it should be matched with them using cosine similarity. At query time the top-k passages are retrieved and the answer is generated from them. Evaluation then covers two things: retrieval, usually as hit rate or recall at k, and the answer itself, through exact match, faithfulness, relevance or citation quality. Chunk size, k and any relevance threshold are adjusted over several iterations. Table 4 shows which techniques appear most often.

Table 4. Techniques used by the existing work, with best reported configuration details

Technique Category	Frequency Count	Efficient Configuration Details
Retrieval-augmented generation architectures (RAG, REALM, FiD, RETRO, in-context RALM, Self-RAG)	08	Few highly relevant passages in the prompt; frozen LLM with in-context retrieval needs no training
Dense bi-encoder retrieval and sentence embeddings (DPR, Sentence-BERT, MiniLM)	05	Cosine similarity over normalised embeddings; compact distilled encoders run on a CPU
Sparse lexical retrieval (BM25)	03	Strong zero-shot baseline across domains
Content-based, cold-start and explainable recommendation	04	Attribute matching avoids dependence on interaction history; explanations from model evidence
Citation and attribution of generated answers	03	Inline source markers; citation recall and precision checked automatically
Approximate nearest-neighbour indexing (HNSW, FAISS)	02	Graph-based HNSW index for low-latency search

Technique Category	Frequency Count	Efficient Configuration Details
Re-ranking and late interaction (BERT cross-encoder, ColBERT)	02	Re-rank a small candidate set from a fast first-stage retriever
LLM-based evaluation (RAGAS, LLM-as-a-judge)	02	Reference-free faithfulness and relevance scores, validated against humans

What stands out is how separate these lines of work are. RAG systems retrieve by similarity and ignore structured constraints. Search systems can filter on attributes, yet they do not produce grounded answers. Recommenders, LLM-based ones in particular, expect interaction histories and tend to explain a ranking after it has been made. We did not find any surveyed work that offers grounded question answering, attribute-filtered search, cold-start recommendation with evidence-based explanations and administrator analytics on top of one knowledge base.

The inputs these systems use are fairly consistent: the query text, passage embeddings and the cosine similarity between them, lexical term statistics, and, for recommendation, item and user attributes. Retrieval is mostly evaluated with exact match and recall at k. Generated answers are judged on faithfulness, relevance, and citation precision and recall, and scores from an LLM judge are becoming more common.

3.2 Research Gaps

- **Blind retrieval:** plain RAG ranks the chunks only using the similarity score. So questions that needs the combination of domain, chapter and the level are answered unreliably.
- **Separate systems:** Question answering, search and recommendation engine are separate pipelines. This should not be categorized into a single one.
- **Cold-start recommendation:** LLM based recommender works on interaction histories done by the users, that small or new communities are not using.
- **Faithful explanations:** explanations are often produced based on the information given, but hallucinations are observed in some of the community chatbots.
- **Refusal and knowledge growth:** few systems refuse when evidence is missing, and fewer still send the unanswered question back so that the knowledge base can grow.
- **Resource feasibility:** most reported systems rely on fine-tuning or large hosted models; there is little evidence about what works on a free tier or an ordinary laptop.

3.3 Formal Models Motivated by the Review

These techniques can be stated compactly, and the proposed system follows the same formulation. Each chunk c and query q is embedded and normalised as $v = E(\cdot)/\|E(\cdot)\|$, giving the cosine distance:

$$d(q, c) = 1 - v_q \cdot v_c \tag{1}$$

Metadata-filtered retrieval keeps up to k nearest chunks, but only those within a distance threshold θ that also satisfy a filter predicate φ over their typed metadata:

$$Ctx(q) = \text{top}_k \{\, c \in V : d(q, c) \le \theta \wedge \varphi(c) \,\} \tag{2}$$

For a generated answer a, let g be its grounded flag and Cit(a) the set of source indices it cites. The answer is accepted only when it is non-empty and cites at least one of the retrieved sources:

$$\text{accept}(a) = g \wedge (a \neq \varnothing) \wedge |Cit(a) \cap \{1, \ldots, k\}| \ge 1 \tag{3}$$

Because new members have no history, peer recommendation uses an attribute-overlap score between member m and candidate p. It rewards a shared domain, common skills, events both have attended, shared interests, the same chapter and institution, and the candidate being a potential mentor:

$$S(m, p) = 2\delta_{dom} + |Sk_m \cap Sk_p| + 1.5|Ev_m \cap Ev_p| + |In_m \cap In_p| + \delta_{ch} + 0.5\delta_{inst} + \delta_{ment} \tag{4}$$

Evaluation uses the retrieval hit rate over a question set Q with gold chunks c^*_q, the answered rate, and judge-assigned faithfulness and relevance over answered questions:

$$\text{HitRate} = |\{\, q \in Q : c^*_q \in Ctx(q) \,\}| / |Q|, \quad \text{AnsweredRate} = |\{\, q \in Q : \text{accept}(a_q) \,\}| / |Q| \tag{5}$$

4. Methodology

Figure 4 gives the overall architecture. Members use a React portal and administrators use a separate console; both call a FastAPI backend [33] that exposes four modules over a REST API, with a token required on administrator routes. Behind the scenes, each and every module uses the RAG pipeline and the information which contains query understanding, metadata-filtered retrieval and grounded generation, which is built on the ChromaDB vector store [34]. Interaction logs and the queue of unanswered questions which are shown to the admin is stored in SQLite database.

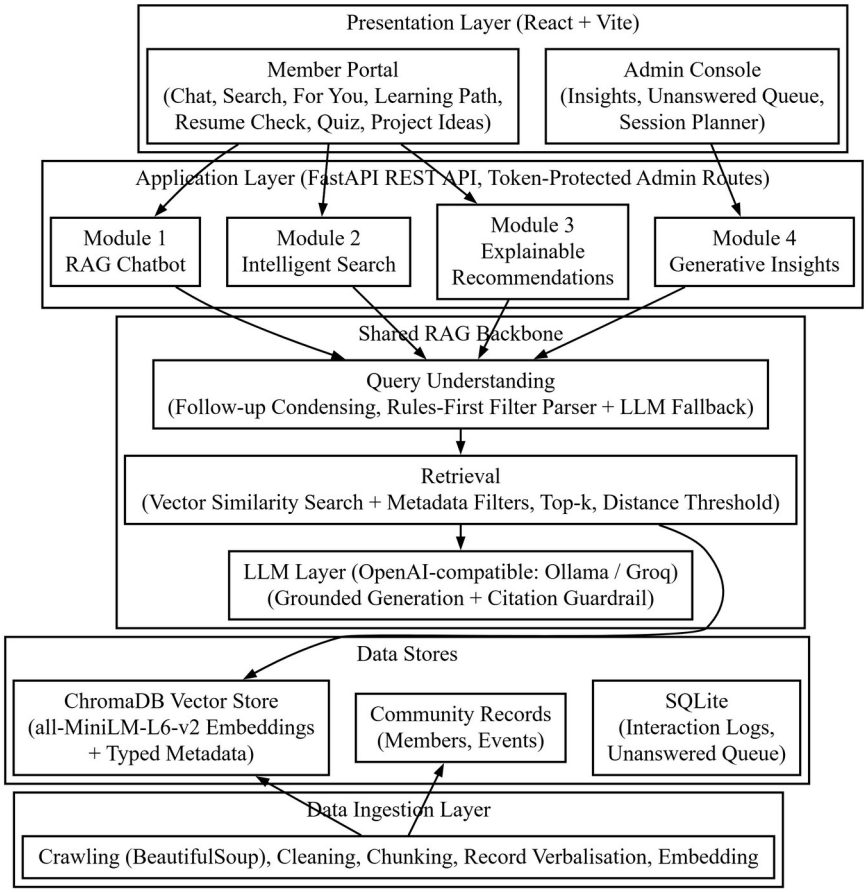


Figure 4. Proposed architecture of community intelligence platform

4.1 Data collection

We have used two types of data collection modules. The first is the unstructured data which is taken from the community websites which is publicly available, starting from the sitemap. The community web pages up to 80 are extracted using the BeautifulSoup waiting one second between the requests and kept 29 pages (108,212 characters) which are about the curriculum, events, hiring, partnerships and FAQs. The second is the structured records. These records are harder to extract, so we wrote a seeded generator. It creates 72 fictional members spread across over six chapter cities, four domains and three levels each with skills, interest and attended events. 32 events are created, which ranges from lab sessions and workshops, hackathons, webinars, open-source sprints and one flagship event per domain. 13 of them were still in the future when we ran the evaluation. The below table describes the result.

Table 5. Description of the knowledge base

Source	Items	Chunks	Mean chunk length (chars)	Metadata fields
Community website	29 pages	169	760	source, URL, title, chunk index
Member records	72	72	249	domain, chapter, level, skills, entity id
Event records	32	32	247	domain, chapter, level, skills, date, event type
Total	**133**	**273**	—	—

4.2 Pre-processing

Web pages were cleaned by dropping scripts and forms and taking text only from leaf blocks such as headings, paragraphs, list items and table cells. This avoids copying the same navigation text into every chunk. The text was then split along paragraph boundaries into chunks of at most about 900 characters, with a 150-character overlap that never cuts a word in half. A chunk's ID is the SHA-1 hash of its URL and position, which makes ingestion idempotent: running it again updates chunks rather than duplicating them. Structured records were verbalised into a sentence such as "X (Student) is a Beginner member of the Cybersecurity community, Pune chapter ... Skills: ... Interests: ..." and stored with their attributes as metadata. One store can therefore serve semantic retrieval and attribute filtering at the same time.

4.3 Embedding and retrieval

We embed chunks and queries with all-MiniLM-L6-v2, a 384-dimensional Sentence-Transformers model small enough to run on a CPU [18, 19], and store them in ChromaDB under a cosine HNSW index [23]. Retrieval follows Eq. (2). For the chatbot we use $k = 4$ and $\theta = 0.65$ and, by default, leave member records out; otherwise short profiles tend to crowd out curriculum content. The more chunks are retrieved from the search and the learning tools. The context is usually empty if nothing passes the threshold and the system refuses and the LLM is not called at all.

4.4 Grounded generation and guardrail

Follow ups are the common problems in the chatbot. The Figure 5 shows that how our model is handling the follow-up questions like "it", "that", "they". The questions are first rewritten into a standalone question using the last six turns. The retrieved chunks are numbered and are fed to the prompt which is gone to the LLM to answer only from them, cite sources inline as [n], keep to five sentences and reply in JSON with the fields grounded, answer, sources and followups. Equation (3) decides whether the answer is accepted. Cheaper models often forget the citation marks. So a grounded answer with no citation can still be credited to one of the top two sources when the source contains at least 35% of the answer's content words. Anything else gets a fallback message. The question joins and goes to the administrator queue, which is shown to the admin side. And that question and answers is logged into the database.

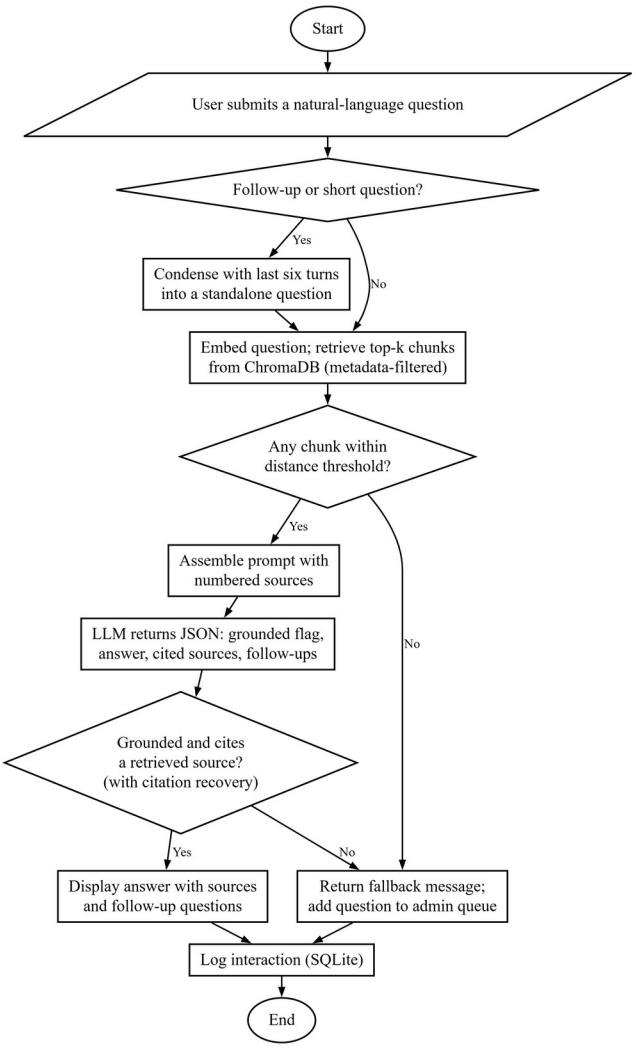


Figure 5. Query-processing workflow of the grounded chatbot

The provider can be swapped as the large language model layers is compatible with any OpenAI endpoint. Two models are tested, gpt-oss-20b on Groq's free tier [35] and qwen2.5:3b, which is served from Ollama [36]. Requests are throttled and retried with the backoff. It is asked for strict JSON output and reply that is not valid in JSON triggers one self-correction prompt.

4.5 Search, recommendation and insights

Search. The search is going through different modules. The first is the query understanding which is handled by the rules. The words are mapped using the phrases like "pentest", "newbie" to domain, level, type and date filters. When no rule is matched, the LLM parser runs and its output is validated against a vocabulary so that it cannot create a new filter value. If the user also sets filter in the interface, those are taken as the priority. The final filters became the predicate φ in Equation (2) and every result states why it matched.

Recommendation. Peers are first scored with Eq. (4). The fifteen best are then re-ranked by 0.7 × normalised score + 0.3 × cosine similarity between the two profile embeddings, and the top five are returned. In the case of the events, the upcoming ones that the member has not attended are considered and are scored on domain, chapter, skills and level and kept if the score reaches 2.5. The modules of the curriculum are retrieved from the vector database for the member's current level and the next one. The LLM handles the score from the evidence which is generated behind the scenes. That writes a single sentence of no more than 30 words. The actual reason for the suggestion is therefore states the actual explanation. If the call of the LLM fails, a "Suggested because: ..." sentence is assembled from the same evidence.

Insights. Embedding and grouping of the logged questions by greedy clustering at a cosine similarity of 0.72 is done which segregates trending questions from the knowledge gaps. Counting of demand per domain and asking the LLM for a summary under 200 words is done. When an administrator answers the logged question, it directly gets fed into the vector database at the next generation so that the next member gets a grounded reply. Several other member tools, learning path, resume skill-gap analyzer, readiness quiz and portfolio project ideas run on the same retrieval layer.

4.6 Implementation and evaluation design

Python 3.11 version is used in the backend with FastAPI [33], ChromaDB [34], Sentence-Transformers and SQLite. BeautifulSoup and Requests are used for the crawling of the websites and PyPDF reads the uploaded resumes. The frontend is built on React with Vite. Every experiment ran on a CPU-only development laptop, always on a copy of the data.

Evaluation was done in four parts. (i) Sixteen functional test cases were run against the REST API through the FastAPI test client. (ii) Following the RAGAS approach [28], an LLM wrote one question for each of 20 randomly sampled web chunks, phrased so that only that chunk could answer it; the chunk serves as the gold label. Each question went through the full pipeline, and gpt-oss-20b, acting as judge, scored faithfulness (0/1) and relevance (1–5) [29]. (iii) Ten multi-constraint member questions were generated, six combining domain, chapter and level and four combining domain, level and interest, with true answers computed directly from the records. An answer counted as correct only if it named exactly the true set of members. We ran these questions through plain vector retrieval over all chunks and through metadata-filtered retrieval, with six retrieved chunks and the same generator in both cases. (iv) Answer generation time was recorded for both LLMs.

4.7 Experimental setup

Synthetic data is being used for the experiment as it lets us build the edge cases and replay them as often as needed without touching the real users' data. The real users' data is confidential, so the synthetic data could be used for the development purpose because the 72 member and 32 event records from Section 4.1 come from a seeded generator, so identical expected answers are expected from it. The crawled website content was taken from a snapshot. Each run also started from a fresh copy of the vector store and SQLite logs, so nothing queued or answered in one experiment could leak into the next. Table 6 lists the setup.

Table 6. Experimental setup

Item	Setting
Hardware	CPU-only development laptop; no GPU
Software	Python 3.11, FastAPI, ChromaDB, Sentence-Transformers, SQLite; React and Vite frontend
Data	29 crawled community website pages (frozen at ingestion); 72 synthetic members and 32 synthetic events from a fixed-seed generator
Knowledge base	273 chunks (169 web, 72 member, 32 event); all-MiniLM-L6-v2, 384 dimensions; cosine HNSW index
LLMs	gpt-oss-20b on the Groq free tier (hosted); qwen2.5:3b through Ollama (local)
Default retrieval	$k = 4$, $\theta = 0.65$; member records excluded from chatbot retrieval
Configurations compared	$\theta \in \{0.50, 0.65, 0.80\}$ with $k = 4$; $k = 6$ with $\theta = 0.65$; plain vector versus metadata-filtered retrieval
Evaluation sets	16 functional and edge-case tests; 20 LLM-generated single-chunk questions; 10 multi-constraint member questions
Isolation	Fresh copy of the vector store and SQLite logs for every run

Edge cases. The synthetic data was used to build edge cases that ensure that every unexpected failure path is handled in the pipeline. It may happen some of the answers end with refusal which are actually out of scope and some of those are in scope. The questions with no chunk inside the vector database cannot be answered from the LLM. This comes under the hallucination. The referential follow-up (TC03) has to be rewritten before retrieval. The multi-constraint question also cover the edge cases situation since many synthetic data share a domain but differ in chapter or the level where similarity search alone is most likely to confuse the records.

4.8 Validation metrics

Table 7 gives each validation metric, how it is computed and the evaluation set it applies to. Retrieval and guardrail behaviour are measured deterministically against gold labels. Answer quality relies on the LLM judge. Relational correctness is checked against ground truth derived from the records, and robustness is the pass rate of the functional and edge-case tests.

Table 7. Validation metrics

Metric	Definition	Evaluation set
Retrieval hit rate	Share of questions whose gold chunk is among the retrieved chunks, Eq. (5)	20 generated questions
Answered rate	Share of questions whose answer passes the guardrail, Eq. (5); its complement is the refusal rate	20 generated questions
Faithfulness	LLM-judge label (0/1): the answer is fully supported by the retrieved sources; mean over answered questions	Answered questions
Answer relevance	LLM-judge score (1–5) of how well the answer addresses the question; mean over answered questions	Answered questions
Exact-set accuracy	Share of questions for which the set of members named equals the true set computed from the records	10 multi-constraint questions
Test pass rate	Share of tests whose output and HTTP status match the expected values	16 test cases
Response time	Wall-clock time of answer generation	Generated answers
Judge–human agreement (planned)	Cohen's κ between LLM-judge and human faithfulness labels; Spearman's ρ for relevance	Answered questions (Section 5.7)

5. Experimental Results and Discussion

5.1 Characteristics of the knowledge base

The make-up of the knowledge base is shown in Figure 6. Web content accounts for 169 of the 273 chunks, averaging 760 characters each, while every one of the 104 verbalised records is a single short chunk of about 250 characters. Members are spread across the four domains (15–21 per domain). Most are Beginner (33) or Intermediate (32), with only 7 Advanced. Events are divided evenly, eight per domain. The roughly threefold difference in length between records and web chunks is one reason member records are excluded from the chatbot's default retrieval.

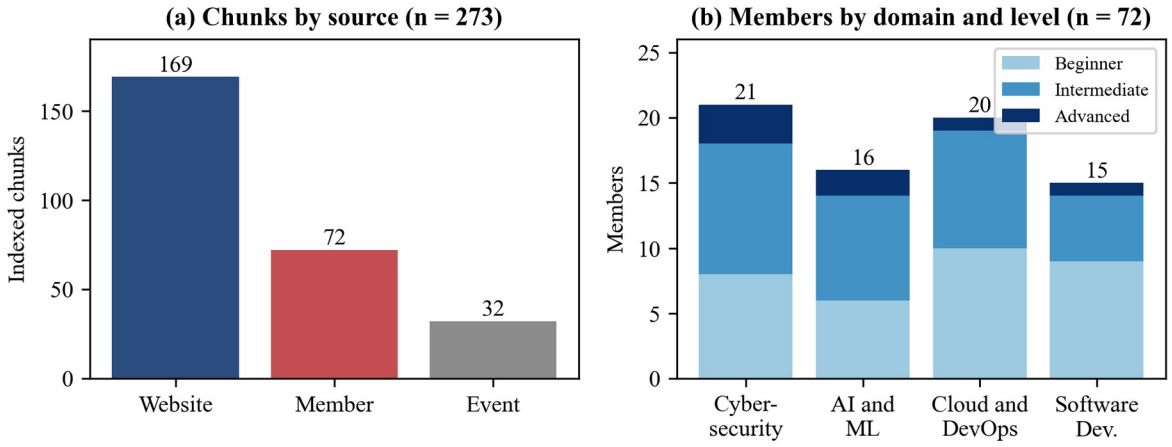


Figure 6. Composition of the knowledge base: (a) chunks by source, (b) members by domain and level

5.2 Results of functional testing

Table 8 lists the sixteen functional tests; fifteen passed. The failure, TC15, happens when the LLM endpoint is unreachable. The client raises a connection error that is never converted into the runtime error the API maps to HTTP 503, so the API returns HTTP 500 instead. Nothing unsafe reaches the user, because no answer is produced, and the fix is a one-line change in the LLM layer.

Table 8. Functional test cases

TC ID	Module	Test Input	Expected Output	Status
TC01	Chatbot	"What does the Cloud & DevOps track cover?"	Grounded answer with at least one citation	Pass
TC02	Chatbot	"What is the capital of France?" (out of scope)	Fallback message; question queued	Pass

TC ID	Module	Test Input	Expected Output	Status
TC03	Chatbot	Follow-up “Who mentors it?” after an AI and ML question	Condensed standalone question; grounded answer	Pass
TC04	Search	Intermediate AI and ML members in Pune	All matching members returned	Pass
TC05	Search	“beginner pentest workshops in Pune”; then interface level = Intermediate	Four filters parsed with reasons; interface filter overrides level	Pass
TC06	Search	LLM parser returns values outside the vocabulary	Invalid values discarded	Pass
TC07	Recommend	Recommendations for a valid member	≤ 5 peers and events (score ≥ 2.5) with explanations	Pass
TC08	Recommend	LLM unavailable during explanation	“Suggested because: …” fallback	Pass
TC09	Learning path	Beginner AI and ML member who knows Python	Ordered steps grounded in the curriculum	Pass
TC10	Resume	Upload larger than 5 MB; unreadable image	HTTP 413; HTTP 422	Pass
TC11	Quiz	Five questions, Cybersecurity, Beginner	Five MCQs with four options, valid answer and explanation	Pass
TC12	Admin	Administrator endpoint without token	HTTP 401	Pass
TC13	Admin	Answer a queued question, then ask it again	Indexed as FAQ; answered with citation	Pass
TC14	Insights	Open the insights overview	Clusters, domain demand and three-section brief	Pass
TC15	LLM layer	LLM endpoint unreachable	HTTP 503 after retries	Fail
TC16	Ingestion	Build the knowledge base twice	Same chunk count (idempotent)	Pass

5.3 Results of grounded question answering

Table 9 and Figure 7(a) give the automated evaluation on 20 questions with the default settings (k = 4, θ = 0.65) and the hosted model. The gold chunk was retrieved for only 60% of questions, yet 80% were answered. The gap comes from repetition on the community website: the same facts appear on several pages, so an equivalent chunk was often enough. Of the 16 accepted answers, 15 were judged faithful (94%), and mean relevance was 4.81/5. Four questions were refused. Two had no chunk within the distance threshold. In the other two, the answer carried no citation and shared too few words with any retrieved source to be credited. The only unfaithful answer came from a question whose gold chunk was missed; the model drew on related passages that did not fully back it up.

Table 9. Results of the automated evaluation (hosted gpt-oss-20b, n = 20)

Configuration	Retrieval Hit Rate	Answered Rate	Faithfulness	Relevance (1–5)
k = 4, θ = 0.65	60%	80%	94%	4.81

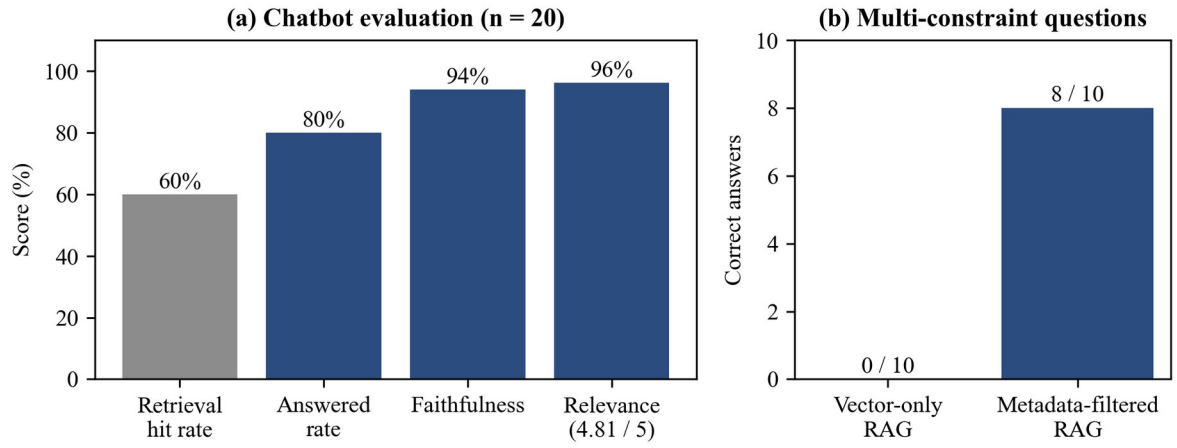


Figure 7. (a) Chatbot evaluation, (b) correct answers on multi-constraint questions

5.4 Results on multi-constraint questions

The two retrieval strategies are compared in Table 10 and Figure 7(b). Plain vector retrieval did not get a single question right. The member chunks closest to each query read almost the same but belong to a different chapter or level, so every generated list either included wrong members or left out correct ones. The retrieval is restricted to the information which is fed to the large language model. So metadata filtering removes that problem. According to the evaluation, eight of ten answers named exactly the right members. Both misses involved interests. Interests appear in the record text but are not a filter field. So it is still dependent on similarity.

Table 10. Plain versus metadata-filtered retrieval on ten multi-constraint questions

Measure	Vector-only RAG	Metadata-filtered RAG
Questions answered with exactly the correct members	0 / 10	8 / 10
Domain + chapter + level questions correct	0 / 6	6 / 6
Domain + level + interest questions correct	0 / 4	2 / 4

5.5 Response time and resource use

Summarization of response time and resource use are there in Table 11. The model which is hosted took 6.9 seconds on average, as it is evaluated on the local model to generate an answer (18 answers). That figure also includes waits which are caused by the free tier of the large language models which gives 8,000 tokens per minute. The local 3 billion parameter model needed about 72 seconds, which is the highest ever recorded for a single answer on the CPU only laptop. Retrieval runs locally in both cases and the entire knowledge base is only a few megabytes.

Table 11. Response time and resource use

Measure	Value
Mean answer generation time, hosted gpt-oss-20b	6.9 s (n = 18, including rate-limit waits)
Answer generation time, local qwen2.5:3b on CPU	≈ 72 s (single run)
Embedding model size and dimension	all-MiniLM-L6-v2, 384 dimensions
Indexed chunks (web / member / event)	273 (169 / 72 / 32)
Hardware	CPU-only laptop; no GPU

5.6 Error analysis and discussion

The failures fall into three groups. The first group is refusals. A question was refused when no chunk was close enough to it. It was also refused when the answer could not be linked to any retrieved source, by a citation or by word overlap. In both cases the guardrail worked as planned. These questions went to the admin queue. They did not reach the users as guesses. The second group is the one unfaithful answer. It was based on only part of the evidence. A re-ranking step [21] or a stricter citation check [26] would likely catch such cases. The third group is the multi-constraint failures. They only happened with attributes that are not filter fields. Making interests a filter field should fix them.

Sensitivity analysis. Two settings control the guardrail. One is the distance threshold. The other is the number of retrieved chunks. With a strict threshold ($\theta = 0.50$), the hit rate fell to 40%. Only 45% of the questions were answered. But every answer that got through was judged faithful. At $\theta = 0.80$, the hit rate went up to 70% and 85% of the questions were answered. Faithfulness stayed at 94%. Six chunks at $\theta = 0.65$ gave the best balance. It gave a 75% hit rate, 85% answered and 100% judged faithful. The cost was longer prompts. Faithfulness never went below 94% in any setting. So the guardrail, not the threshold, keeps unsupported answers away from users. The threshold mostly decides how many questions are answered and how many are refused.

Now back to the research questions. For RQ1, the answer is yes for this knowledge base. A free hosted model answered 80% of the questions with 94% judged faithfulness. It refused the rest instead of making up answers. For RQ2, plain vector RAG got zero questions right. With metadata filtering, eight out of ten were right. For RQ3, all four modules and the extra learning tools used one retrieval layer and one vector store. The functional tests showed that search, recommendation, the explanation fallback, the question queue and insights all worked end to end.

These results have some limits. The evaluation sets are small, with 20 generated and 10 relational questions. The member and event records are synthetic. Faithfulness and relevance were scored by an LLM judge. This judge is not yet checked against human ratings. Also, the hosted response times were slower because of free-tier rate limits.

5.7 Validation of results and planned human evaluation

The LLM is not validating several results. Each generated question has a gold label, the chunk it was written from. So the retrieval hit rate is exact. Exact-set accuracy involves no judgement as expected answers to the multi-constraint questions were computed straight from the structured records. The functional and edge-case tests compare outputs and HTTP status codes with fixed expected values. And because the synthetic data come from a fixed seed, anyone rerunning an experiment gets the same ground truth.

Faithfulness and relevance often comes under two different filters. They were scored by a single LLM judge. And these judges are given a strong system prompt so that they will be highly relevant to validate the model. The same model is used to generate the responses also. We have therefore designed a human validation of the judge to be carried out. From the human based knowledge, whenever the answer gets generated from the LLM, that will be matched alongside the information which is fed into the database by the human only. That will be finalized. And if there are some of the problems, the information fed in the database will be directly edited. Two authors will label every accepted answer independently for faithfulness (0/1) and relevance (1–5) using the judge's rubric. Retrieved sources will be judged by the humans so that the settling of these agreements will be done. Agreement will be measured with Cohen's κ for faithfulness and Spearman's rank correlation for relevance and will be treated the judge as reliable if κ reaches 0.6. The protocol will then be repeated with a second judge, for example the locally served qwen2.5:3b, to see whether scores depend on the judging model. RAGAS is implemented so to add the context precision and context recall in the validation pipeline as it gives the faithfulness and the relevance score a bit strong grip [28].

6. Conclusion and Future Work

Skill-development communities keep their knowledge in many places, like records and web pages. An LLM alone cannot answer questions about this knowledge in a reliable way. We reviewed 36 documents from 2002 to 2025. We found that RAG, retrieval, grounding, evaluation and recommendation methods have grown on separate tracks. No reviewed system brings grounded question answering, filtered search, cold-start recommendation and admin analytics together on one knowledge base.

We built such a platform ourselves. It runs on one metadata-filtered RAG backbone with ChromaDB. The refusal-first guardrail made sure every answer could be traced back to a source. With the hosted model, 80% of the 20 questions were answered. Of these, 94% were judged faithful and the mean relevance was 4.81 out of 5. The rest were refused. Metadata filtering got eight of ten multi-constraint questions exactly right. Plain vector RAG got none. Fifteen of sixteen functional tests passed. These results support our main idea. One lightweight RAG backbone, on a free-tier or local model, can run many community features at once.

Next, we will check the LLM judge against human ratings on a bigger, hand-made question set. We will also add interests and other attributes as filter fields. We will add a cross-encoder re-ranking step and fix the TC15 error mapping. Later, we want to test the system with real member records and live usage logs. We also plan to add multilingual and mobile access and to use the platform in more chapters.

Data availability

The website contents is extracted from the websites which is publicly available. It's hard to find the user data as the confidential information isn't listed publicly, so we have to generate the synthetic data. The source code and evaluation questions are available from the corresponding author on reasonable request.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[2] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019, pp. 4171–4186.

[3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin et al., "Training Language Models to Follow Instructions with Human Feedback," Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022.

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, 2023, pp. 1–38.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[7] A. I. Schein, A. Popescul, L. H. Ungar and D. M. Pennock, “Methods and Metrics for Cold-Start Recommendations,” Proc. 25th ACM SIGIR Conference on Research and Development in Information Retrieval, 2002, pp. 253–260.

[8] K. Guu, K. Lee, Z. Tung, P. Pasupat and M.-W. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” Proc. 37th International Conference on Machine Learning (ICML), PMLR 119, 2020, pp. 3929–3938.

[9] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” Proc. 16th Conference of the European Chapter of the ACL (EACL), 2021, pp. 874–880.

[10] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican et al., “Improving Language Models by Retrieving from Trillions of Tokens,” Proc. 39th International Conference on Machine Learning (ICML), PMLR 162, 2022, pp. 2206–2240.

[11] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown and Y. Shoham, “In-Context Retrieval-Augmented Language Models,” Transactions of the Association for Computational Linguistics, vol. 11, 2023, pp. 1316–1331.

[12] K. Shuster, S. Poff, M. Chen, D. Kiela and J. Weston, “Retrieval Augmentation Reduces Hallucination in Conversation,” Findings of the Association for Computational Linguistics: EMNLP, 2021, pp. 3784–3803.

[13] A. Asai, Z. Wu, Y. Wang, A. Sil and H. Hajishirzi, “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,” International Conference on Learning Representations (ICLR), 2024.

[14] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang and H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv:2312.10997, 2023.

[15] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” Transactions of the Association for Computational Linguistics, vol. 12, 2024, pp. 157–173.

[16] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, 2009, pp. 333–389.

[17] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen and W. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[18] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[19] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 5776–5788.

[20] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” Proc. 43rd ACM SIGIR Conference on Research and Development in Information Retrieval, 2020, pp. 39–48.

[21] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” arXiv:1901.04085, 2019.

[22] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava and I. Gurevych, “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” NeurIPS Datasets and Benchmarks Track, 2021.

[23] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, 2020, pp. 824–836.

[24] J. Johnson, M. Douze and H. Jégou, “Billion-Scale Similarity Search with GPUs,” IEEE Transactions on Big Data, vol. 7, no. 3, 2021, pp. 535–547.

[25] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim et al., “WebGPT: Browser-Assisted Question-Answering with Human Feedback,” arXiv:2112.09332, 2021.

[26] T. Gao, H. Yen, J. Yu and D. Chen, “Enabling Large Language Models to Generate Text with Citations,” Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 6465–6488.

[27] F. Petroni, A. Piktus, A. Fan, P. Lewis, M. Yazdani, N. De Cao et al., “KILT: A Benchmark for Knowledge Intensive Language Tasks,” Proc. NAACL-HLT, 2021, pp. 2523–2544.

[28] S. Es, J. James, L. Espinosa-Anke and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” Proc. 18th Conference of the European Chapter of the ACL (EACL): System Demonstrations, 2024, pp. 150–158.

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, vol. 36, 2023.

[30] P. Lops, M. de Gemmis and G. Semeraro, “Content-based Recommender Systems: State of the Art and Trends,” in Recommender Systems Handbook, F. Ricci, L. Rokach, B. Shapira and P. B. Kantor, Eds. Springer, 2011, pp. 73–105.

[31] Y. Zhang and X. Chen, “Explainable Recommendation: A Survey and New Perspectives,” Foundations and Trends in Information Retrieval, vol. 14, no. 1, 2020, pp. 1–101.

[32] Z. Zhao, W. Fan, J. Li, Y. Liu, X. Mei, Y. Wang, Z. Wen, F. Wang, X. Zhao, J. Tang and Q. Li, “Recommender Systems in the Era of Large Language Models (LLMs),” IEEE Transactions on Knowledge and Data Engineering, vol. 36, no. 11, 2024, pp. 6889–6907.

[33] S. Ramírez, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com

[34] Chroma, “Chroma: The Open-Source AI Application Database – Documentation.” [Online]. Available: https://docs.trychroma.com

[35] OpenAI, “gpt-oss-120b & gpt-oss-20b Model Card,” Technical report, Aug. 2025.

[36] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.